

Beyond MIME: Detecting AI-Generated Python Artifacts Through Structural Forensic Analysis

Pranav Sankar
Liberty High School

Renton, WA
pranavsankar614@gmail.com

Abstract—Traditional digital forensic tools rely heavily on static MIME checks to classify and validate source code files within automated processing pipelines. However, the rapid proliferation of Generative AI has created a severe vulnerability: Large Language Models (LLMs) synthesize synthetically perfect Python code that effortlessly bypasses standard MIME filters while masking their non-human origin. This paper addresses this blind spot by examining the structural anomalies present in synthetic Python code compared to human-written controls. We introduce the MIME Detection and Anomaly Resolution (MIME-DAR) framework, a multi-tiered inspection pipeline designed to detect AI-generated Python source code by profiling code stylometry and low-level environmental formatting artifacts. On a proof-of-concept dataset of 60 files (30 human, 30 AI-generated), structural stylometry features trended in the hypothesized direction but did not reach statistical significance at $\alpha = 0.05$, while environmental artifact features showed little to no discriminative signal. A logistic regression classifier combining all features achieved 72.2% accuracy but a 55.6% false positive rate on human-authored files, indicating the current feature set is not yet reliable enough for forensic deployment. We report these results in full, including negative findings, and outline the scope limitations that future work must address.

Index Terms—digital forensics, MIME, artificial intelligence, Python, code analysis, cybersecurity

I. INTRODUCTION

Traditional digital forensics relies on fixed, pattern-based tools to analyze metadata and file structures to verify digital evidence. In modern security systems, Multipurpose Internet Mail Extensions (MIME) classifications are used as the primary gatekeeper to quickly identify and sort incoming code files. However, the rapid evolution of Generative AI has created a major vulnerability in this approach. Because advanced language models create Python source files that are synthetically perfect, these files easily slide past the traditional signature checks administered by MIME filters. This creates a serious blind spot: automated tools flag AI-generated Python scripts as completely authentic, even though they lack the expected structural footprints generated by human software development.

This paper investigates this problem by analyzing LLM-generated Python source code against local human control samples. Based on the structural patterns discovered in these scripts, this study outlines an analysis of synthetic footprints and proposes the MIME Detection and Anomaly Resolution (MIME-DAR) framework. MIME-DAR is a multi-tiered inspection model designed to catch the subtle structural differ-

ences and stylistic anomalies that standard MIME detectors inherently miss.

This study is a small-scale proof of concept, not a validated production system. Its purpose is to test a specific hypothesis that human and AI-authored Python files differ in measurable structural ways using real data and standard statistical methods, and to report honestly what does and does not hold up under that test.

II. RELATED WORK

The intersection of file verification, generative AI, and digital forensics is becoming a major focus in cybersecurity. Because modern security systems rely heavily on quick automated checks to sort and verify incoming source code, recent research has started looking into how these systems can be bypassed and how we can better detect synthetic data.

The vulnerabilities in traditional file type checks are well documented. Udomwongsa [1] showed how attackers can bypass security software by exploiting the way systems identify file types. This research highlights that traditional security tools often look only at basic signatures or headers without checking the structural style and internal composition of the file. If a file satisfies basic file-type criteria, it can easily slip past initial security filters.

This issue becomes even more complicated when dealing with source code generated by large language models (LLMs). Yin et al. [2] studied the challenge of identifying AI-generated code, especially when it includes subtle edits like changed variable names or altered structures. To solve this, they created CodeGPTSensor+, a system that uses an adversarial module called MIST to simulate structural alterations. By training their model on these varied code structures, they were able to detect AI-generated code more accurately. Their work proves that standard security tools struggle against synthetically perfect code, showing that we need more adaptive detection methods to find the hidden, non-human patterns in AI-generated files.

On a broader scale, generative AI introduces unique challenges for the entire field of digital forensics. Walker and Baggili [3] looked closely at the artifacts left behind by AI tools while exploring how investigators can use AI as an analytical tool. Their research emphasizes that AI-generated files lack the typical footprints left by human development, such as natural formatting variations, specific IDE artifacts, or subtle syntax irregularities. Because standard forensic tools

are not designed to look for the absence of human traits, investigators need new frameworks specifically built to flag these perfectly generated synthetic scripts.

While existing research covers file validation evasion, AI code detection, and forensic artifacts, there remains a critical gap in how Python source code interacts with MIME classification systems. Current security tools trust MIME classifications to filter files, assuming that if a file passes standard syntax validation, the content is human-authored. This paper addresses this specific blind spot by introducing the MIME Detection and Anomaly Resolution (MIME-DAR) framework tailored specifically for Python source code analysis, and by empirically testing it rather than presenting it as a purely theoretical design.

III. METHODOLOGY

To address the forensic vulnerabilities caused by synthetically perfect Python code, this section introduces the MIME Detection and Anomaly Resolution (MIME-DAR) framework. Traditional security tools fail because they only check whether a script complies with valid Python syntax definitions. MIME-DAR solves this issue by treating structural hyper-perfection and the absence of ordinary human formatting variation as core anomalies. The framework operates as a three-tiered inspection pipeline. Tier 1 acts as a fast pass/fail gate; Tiers 2 and 3 extract measurable structural features that feed into a statistical classifier.

A. Dataset

The dataset consists of 60 Python files: 30 human-written files collected from public GitHub repositories, and 30 AI-generated files produced by a single large language model (Claude). All AI files were generated from short, varied prompts (e.g., "write a function that performs binary search") to cover a range of common programming tasks comparable in scope to the human sample. Human files were used unmodified as scraped, preserving their original formatting.

This dataset is a proof-of-concept sample, not a large-scale corpus, and its size and composition directly shape which results below could reach statistical significance.

B. Framework Architecture and Tiered Pipeline

The operational pipeline of MIME-DAR is divided into three distinct validation stages, moving from fast, low-resource checks to deep structural profiling:

Tier 1 (Canonical Verification): This layer acts as the initial check, mimicking traditional MIME detection. It validates file extensions, sniffs the file's MIME type, confirms the file is non-empty, and confirms the file parses as syntactically valid Python via Abstract Syntax Tree (AST) compilation. This tier functions as a binary gate, not a scored feature: files that fail are excluded from further analysis, and files that pass proceed to Tier 2. All 60 files in this study's dataset passed Tier 1 with zero exclusions.

Tier 2 (Structural Stylometry): This tier analyzes the spatial layout and comment habits of Python source files,

measuring structural consistency against typical human coding habits and flagging files that display mathematical perfection in their formatting.

Tier 3 (Environmental Artifact Profiling): This tier scans plain-text source files for low-level traces left by real editing environments, rather than application-level metadata. It evaluates line-ending consistency, trailing whitespace, and the presence of a final newline at end-of-file—small formatting habits that vary with human editing tools but are typically normalized in LLM output. This tier was redesigned from an earlier Excel-metadata-based concept after the study's empirical scope was narrowed to Python source files only; it is not a validated tier from prior forensic literature but an original design tested here for the first time.

C. Feature Extraction for Python Source Code

MIME-DAR relies on targeted extraction rules tailored specifically to Python code structures. The analyzed anomalies are grouped into two tiers of measurable features:

1) Tier 2 Structural Stylometry:

- **Indentation Variance:** the statistical variance in leading-whitespace width across all indented lines in a file. Human developers frequently mix spacing or drift in indentation width across a large file, whereas LLM output tends toward uniform, low-variance indentation.
- **Comment Placement Ratio:** the fraction of comments that appear directly above a function or class definition. A high ratio suggests uniform, boilerplate-style commenting associated with LLM output.

2) Tier 3 Environmental Artifacts:

- **Mixed Line Ending Ratio:** the fraction of newlines belonging to the minority line-ending style (CRLF vs. LF) within a file.
- **Trailing Whitespace Ratio:** the fraction of non-empty lines ending in trailing spaces or tabs.
- **Missing Final Newline:** a binary indicator of whether the file fails to end with a newline character.

D. Statistical and Classification Methodology

Rather than assigning tier weights manually, this study replaces the theoretical weighting scheme with two empirical statistical procedures applied directly to the extracted features.

First, a Welch's t-test (which does not assume equal variance between groups) was run independently on each feature, comparing the human-file group against the AI-file group, to test whether each feature alone statistically separates the two classes.

Second, the five features were combined using a logistic regression classifier trained on a stratified 70/30 train-test split (42 training files, 18 test files). Feature values were standardized prior to training so that the resulting model coefficients are directly comparable in magnitude. These learned coefficients serve as the empirical Weighted Anomaly Scoring Vector (WASV), replacing the static, hand-assigned weights (w_1, w_2, w_3) originally proposed in this framework's design phase.

Classifier performance was evaluated using accuracy, precision, recall, and false positive rate on the held-out test set.

IV. RESULTS

A. Tier 1 Gate

All 60 files in the dataset (30 human, 30 AI-generated) passed Tier 1 canonical verification with zero exclusions. No files were corrupted, mislabeled, or syntactically invalid, so the full dataset proceeded to Tier 2 and Tier 3 feature extraction.

B. Feature-Level Hypothesis Testing

Table I reports the Welch’s t-test results for each of the five extracted features.

TABLE I
WELCH’S *t*-TEST RESULTS BY FEATURE

Feature	Human Mean	AI Mean	p-value
Indentation Variance	10.789	6.705	0.063
Comment Placement Ratio	0.070	0.000	0.059
Mixed Line Ending Ratio	0.000	0.000	-
Trailing Whitespace Ratio	0.001	0.000	0.180
Missing Final Newline	0.000	0.000	-

Indentation variance and comment placement ratio both trended in the hypothesized direction: human files showed higher indentation variance and a higher rate of non-boilerplate comment placement than AI-generated files. Neither reached statistical significance at the conventional $\alpha = 0.05$ threshold ($p = 0.063$ and $p = 0.059$, respectively), though both fell close enough to the threshold to suggest the effect may be real but underpowered by this study’s sample size of 30 files per class.

Trailing whitespace ratio was also directionally consistent (human mean $\hat{>}$ AI mean) but far from significant ($p = 0.180$)—only 2 of the 30 human files contained any trailing whitespace at all, limiting this feature’s ability to separate the groups in this sample.

Mixed line ending ratio and missing final newline both showed zero variance across the entire dataset—every file in both classes used a single, consistent line-ending style and ended with a final newline. Because there is no variance to test, a t-test statistic cannot be computed for these features (reported as - in Table I). This is a genuine null result: these two Tier 3 features carried no discriminative information in this dataset, likely because common development tooling (e.g., version control line-ending normalization, standard editor save behavior) erases these artifacts regardless of a file’s human or AI origin.

C. Classification Performance

Table II reports the logistic regression classifier’s learned weights and evaluation metrics on the held-out test set (18 files: 9 human, 9 AI).

The classifier assigned a learned weight of exactly 0.000 to both mixed line ending ratio and missing final newline, independently confirming the t-test finding that these features carry no usable signal in this dataset. Comment placement

TABLE II
LOGISTIC REGRESSION RESULTS (STANDARDIZED WEIGHTS)

Feature	Learned Weight
Indentation Variance	-0.268
Comment Placement Ratio	-1.021
Mixed Line Ending Ratio	0.000
Trailing Whitespace Ratio	-0.644
Missing Final Newline	0.000

Metric	Value
Accuracy	0.722
Precision	0.643
Recall	1.000
False Positive Rate	0.556

ratio received the largest-magnitude weight, indicating it was the most influential feature in the model despite not individually reaching significance in isolation—a t-test and a multivariate classifier answer different questions, and a feature can contribute to a combined decision boundary without clearing significance on its own.

The classifier achieved 72.2% overall accuracy and perfect recall (1.000), correctly identifying all 9 AI-generated files in the test set. However, precision was only 0.643, and the false positive rate was 0.556: of the 9 human files in the test set, 5 were incorrectly classified as AI-generated. The confusion matrix was 4 true negatives, 5 false positives, 0 false negatives, and 9 true positives.

This result pattern—high recall, weak precision, elevated false positive rate—indicates the current feature set is biased toward over-flagging files as synthetic rather than under-flagging them. For a forensic tool whose stated motivation is preventing employees from being wrongly implicated by AI-generated artifacts, a high false positive rate on human-authored code is a serious practical limitation, not a minor statistical footnote.

V. LIMITATIONS

Several limitations constrain how these results should be interpreted.

Sample size. With only 30 files per class, the study is underpowered to detect moderate effect sizes at conventional significance thresholds. Both Tier 2 features showed p-values just above 0.05, which is consistent with a real but modest effect that a larger sample could confirm, but this study cannot make that claim on the current data.

Single-model AI generation. All AI-generated files were produced by a single LLM (Claude). The classifier and statistical tests in this study therefore measure how distinguishable that model’s Python output is from human code, not AI-generated Python code in general. Detection performance may differ substantially against code generated by other models.

Tier 3 feature validity. Two of the three Tier 3 features (mixed line ending ratio, missing final newline) showed zero variance across the entire dataset and provided no discriminative value. This may reflect a genuine absence of signal in these artifacts, or it may reflect that this study’s data

collection and generation process (e.g., GitHub’s typical line-ending normalization, consistent AI-generation environment) inadvertently erased variation that might exist in a broader, more heterogeneous dataset. This tier, as implemented, should be considered exploratory rather than validated.

Elevated false positive rate. A 55.6% false positive rate on human files makes the current classifier unsuitable for real forensic deployment, where incorrectly flagging a human author as an AI source carries direct reputational and security consequences. This is the most important limitation of the current framework and the clearest priority for future work.

Scope. This study is limited to Python source files. The original MIME-DAR concept proposed extending analysis to HTML, Java, Excel, and plain-text formats; those extensions remain untested and are left to future work.

VI. CONCLUSION

This paper presented MIME-DAR, a multi-tiered forensic framework for detecting AI-generated Python source code, and tested it empirically on a proof-of-concept dataset of 30 human and 30 AI-generated files. Structural stylometry features (indentation variance, comment placement) trended in the hypothesized direction but did not reach statistical significance given this study’s sample size, while two of three environmental artifact features showed no discriminative signal at all.

A logistic regression classifier combining all five features achieved 72.2% accuracy but an elevated false positive rate on human-authored files, indicating the framework in its current form is not yet reliable enough for practical forensic use.

These results are reported in full, including the negative and inconclusive findings, because an honest account of what does not yet work is a necessary part of building a framework that eventually does. The clearest paths forward are a larger and more diverse dataset, AI-generated samples drawn from multiple language models rather than one, and additional structural features—particularly at the Abstract Syntax Tree level—that were proposed in this framework’s original design but not yet implemented or tested. Pursuing this kind of large-scale, statistically rigorous, and linguistically informed detection research is precisely the direction this project is intended to grow into at the undergraduate level, through coursework and lab work spanning cybersecurity, statistics, and natural language processing.

REFERENCES

- [1] C. Udomwongsa, "Evading Antivirus Detection by Abusing File Type Identification," Dartmouth College, Computer Science Senior Thesis, 2024. [Online]. Available: https://digitalcommons.dartmouth.edu/cs_senior_theses/44/
- [2] X. Yin, X. Li, C. Ni, X. Xu, and X. Yang, "Detecting LLM-generated Code with Subtle Modification by Adversarial Training," arXiv preprint arXiv:2507.13123, 2025.
- [3] C. J. Walker, "Digital Forensics and AI: Artifact Analysis and Using AI in the Forensics Domain," Ph.D. dissertation, Louisiana State University, Baton Rouge, LA, USA, 2025. [Online]. Available: https://repository.lsu.edu/gradschool_dissertations/6884/